

Working with Arrays in PHP

Arrays in PHP are essential for storing and managing multiple values efficiently. They allow developers to organize data logically and manipulate it using loops and iterations.

1. Storing Data in Arrays

PHP provides three types of arrays:

1. **Indexed Arrays** – Arrays with numeric keys.
2. **Associative Arrays** – Arrays with named keys.
3. **Multidimensional Arrays** – Arrays containing multiple arrays.

Example: Indexed Array

```
$fruits = array("Apple", "Banana", "Cherry");  
echo $fruits[0]; // Output: Apple
```

Example: Associative Array

```
$person = array("Name" => "John", "Age" => 30, "City" => "New York");  
echo $person["Name"]; // Output: John
```

Example: Multidimensional Array

```
$students = array(  
    array("Alice", 22, "A"),  
    array("Bob", 21, "B"),  
    array("Charlie", 23, "A+")  
);  
echo $students[1][0]; // Output: Bob
```

2. Processing Arrays with Loops and Iterations

PHP provides different looping structures to iterate over arrays.

Using for Loop (Indexed Arrays)

```
$colors = array("Red", "Green", "Blue");  
  
for ($i = 0; $i < count($colors); $i++) {  
    echo $colors[$i] . "<br>";  
}
```

Using foreach Loop (Associative Arrays)

```
$person = array("Name" => "John", "Age" => 30, "City" => "New York");  
  
foreach ($person as $key => $value) {
```

```
    echo "$key: $value <br>";  
}
```

Using while Loop

```
$numbers = array(10, 20, 30, 40);  
$i = 0;  
  
while ($i < count($numbers)) {  
    echo $numbers[$i] . "<br>";  
    $i++;  
}
```

Using do-while Loop

```
$i = 0;  
do {  
    echo $numbers[$i] . "<br>";  
    $i++;  
} while ($i < count($numbers));
```

3. Useful Array Functions

PHP provides built-in functions to manipulate arrays effectively.

Sorting Arrays

```
sort($colors); // Ascending order  
rsort($colors); // Descending order
```

Counting Elements

```
echo count($colors); // Output: 3
```

Checking if a Key Exists

```
if (array_key_exists("Age", $person)) {  
    echo "Key exists!";  
}
```

Merging Arrays

```
$array1 = array(1, 2, 3);  
$array2 = array(4, 5, 6);  
$merged = array_merge($array1, $array2);
```

Removing Elements

```
unset($colors[1]); // Removes "Green"
```

PHP Programs Using Arrays

1. Create and Display an Indexed Array

```
<?php
// Creating an indexed array
$fruits = array("Apple", "Banana", "Cherry", "Mango");

// Displaying array elements using a loop
echo "Fruits List:<br>";
foreach ($fruits as $fruit) {
    echo $fruit . "<br>";
}
?>
```

Output:

Fruits List:
Apple
Banana
Cherry
Mango

2. Sum of All Elements in an Array

```
<?php
$numbers = array(10, 20, 30, 40, 50);

// Calculate sum using array_sum()
$sum = array_sum($numbers);

echo "Sum of array elements: $sum";
?>
```

Output:

Sum of array elements: 150

3. Find Maximum and Minimum Values in an Array

```
<?php
$values = array(5, 10, 2, 8, 15, 3);

$max = max($values);
$min = min($values);

echo "Maximum Value: $max <br>";
echo "Minimum Value: $min";
?>
```

Output:

Maximum Value: 15

Minimum Value: 2

4. Reverse an Array

```
<?php
$numbers = array(1, 2, 3, 4, 5);

// Reversing array using array_reverse()
$reversed = array_reverse($numbers);

echo "Reversed Array: ";
print_r($reversed);
?>
```

Output:

Reversed Array: Array ([0] => 5 [1] => 4 [2] => 3 [3] => 2 [4] => 1)

5. Remove Duplicate Elements from an Array

```
<?php
$values = array(10, 20, 10, 30, 40, 20, 50);

// Remove duplicates using array_unique()
$uniqueValues = array_unique($values);

echo "Unique Array: ";
print_r($uniqueValues);
?>
```

Output:

Unique Array: Array ([0] => 10 [1] => 20 [3] => 30 [4] => 40 [6] => 50)

6. Sorting an Array (Ascending and Descending)

```
<?php
$numbers = array(50, 20, 40, 10, 30);

// Sorting in ascending order
sort($numbers);
echo "Ascending Order: ";
print_r($numbers);

echo "<br>";

// Sorting in descending order
```

```

rsort($numbers);
echo "Descending Order: ";
print_r($numbers);
?>

```

Output:

Ascending Order: Array ([0] => 10 [1] => 20 [2] => 30 [3] => 40 [4] => 50)
 Descending Order: Array ([0] => 50 [1] => 40 [2] => 30 [3] => 20 [4] => 10)

7. Merge Two Arrays

```

<?php
$array1 = array("Red", "Green", "Blue");
$array2 = array("Yellow", "Pink", "Purple");

// Merging arrays using array_merge()
$mergedArray = array_merge($array1, $array2);

echo "Merged Array: ";
print_r($mergedArray);
?>

```

Output:

Merged Array: Array ([0] => Red [1] => Green [2] => Blue [3] => Yellow [4] => Pink [5] => Purple)

8. Find the Frequency of Elements in an Array

```

<?php
$numbers = array(1, 2, 2, 3, 3, 3, 4, 4, 4, 4);

// Count values using array_count_values()
$frequency = array_count_values($numbers);

echo "Frequency of elements:<br>";
foreach ($frequency as $key => $value) {
    echo "$key occurs $value times.<br>";
}
?>

```

Output:

Frequency of elements:
 1 occurs 1 times.
 2 occurs 2 times.
 3 occurs 3 times.
 4 occurs 4 times.

9. Check if an Element Exists in an Array

```
<?php
$colors = array("Red", "Green", "Blue", "Yellow");

$search = "Green";

if (in_array($search, $colors)) {
    echo "$search is present in the array.";
} else {
    echo "$search is not present in the array.";
}
?>
```

Output:

Green is present in the array.

10. Remove a Specific Element from an Array

```
<?php
$fruits = array("Apple", "Banana", "Cherry", "Mango");

// Remove "Banana"
unset($fruits[1]);

// Re-index the array
$fruits = array_values($fruits);

print_r($fruits);
?>
```

Output:

Array ([0] => Apple [1] => Cherry [2] => Mango)

11. Program to Search for an Element in an Array

```
<?php
$colors = array("Red", "Blue", "Green", "Yellow");

$search = "Green";
if (in_array($search, $colors)) {
    echo "$search is found in the array.";
} else {
    echo "$search is not in the array.";
}
?>
```

Output:

Green is found in the array.

Using Arrays with Forms in PHP

PHP allows handling arrays in forms, processing user input, and performing various operations with array functions.

1. Creating a Form to Collect Data in an Array

Here's an example where users enter their favorite fruits, and we store them in an array.

HTML Form:

```
<!DOCTYPE html>
<html>
<head>
  <title>Favorite Fruits</title>
</head>
<body>
  <form method="post">
    Enter your favorite fruits (comma-separated):
    <input type="text" name="fruits">
    <input type="submit" name="submit" value="Submit">
  </form>

  <?php
  if (isset($_POST['submit'])) {
    $fruits = explode(",", $_POST['fruits']); // Convert input to an array
    echo "<h3>Your Favorite Fruits:</h3>";
    echo "<ul>";
    foreach ($fruits as $fruit) {
      echo "<li>" . trim($fruit) . "</li>";
    }
    echo "</ul>";
  }
  ?>
</body>
</html>
```

Explanation:

- The form takes a comma-separated list of fruits.
- `explode(",", $_POST['fruits'])` converts the input string into an array.
- `trim($fruit)` removes any extra spaces.

Working with Array Functions

PHP provides built-in functions for handling arrays efficiently.

2. Common Array Functions with Example

Sorting an Array

```
<?php
$numbers = array(34, 12, 45, 78, 9);
sort($numbers); // Sort in ascending order
print_r($numbers);
?>
```

Output:

Array ([0] => 9 [1] => 12 [2] => 34 [3] => 45 [4] => 78)

Searching for an Element in an Array

```
<?php
$colors = array("Red", "Blue", "Green", "Yellow");
if (in_array("Green", $colors)) {
    echo "Green is found in the array.";
}
?>
```

Output:

Green is found in the array.

Finding Maximum and Minimum Values

```
<?php
$values = array(10, 20, 30, 5, 40);
echo "Max: " . max($values) . "<br>";
echo "Min: " . min($values);
?>
```

Output:

Max: 40

Min: 5

Merging Two Arrays

```
<?php
$array1 = array("Apple", "Banana");
$array2 = array("Mango", "Orange");
$merged = array_merge($array1, $array2);
print_r($merged);
?>
```

Output:

Array ([0] => Apple [1] => Banana [2] => Mango [3] => Orange)

Removing Duplicates


```
<?php
$numbers = array(10, 20, 10, 30, 20, 40);
$unique = array_unique($numbers);
print_r($unique);
?>
```

Output:

Array ([0] => 10 [1] => 20 [3] => 30 [5] => 40)

Working with Date and Time in PHP

PHP provides the date() function and DateTime class to handle date and time operations.

3. Displaying the Current Date and Time

```
<?php
echo "Today's Date: " . date("Y-m-d") . "<br>";
echo "Current Time: " . date("H:i:s") . "<br>";
?>
```

Output (Example):

Today's Date: 2025-04-02
Current Time: 14:30:45

4. Formatting Date and Time

```
<?php
echo "Formatted Date: " . date("l, F j, Y") . "<br>";
?>
```

Output:

Formatted Date: Tuesday, April 2, 2025

5. Getting the Current Timestamp

```
<?php
echo "Current Timestamp: " . time();
?>
```

Output:

Current Timestamp: 1712063400

6. Converting Timestamp to Date

```
<?php
$timestamp = 1712063400;
```

```
echo "Date from Timestamp: " . date("Y-m-d H:i:s", $timestamp);  
?>
```

Output:

Date from Timestamp: 2025-04-02 14:30:00

7. Calculating Date Differences

```
<?php  
$date1 = new DateTime("2025-01-01");  
$date2 = new DateTime("2025-04-02");  
$diff = $date1->diff($date2);  
echo "Difference: " . $diff->days . " days";  
?>
```

Output:

Difference: 91 days

8. Displaying Server Time with Timezone

```
<?php  
date_default_timezone_set("Asia/Kolkata");  
echo "Current Time in India: " . date("H:i:s");  
?>
```

Output (Example for IST):

Current Time in India: 19:45:30

String Manipulation in PHP

String manipulation is an essential part of PHP, allowing developers to create, access, and modify strings efficiently. PHP provides various built-in functions to handle string operations.

1. Creating and Accessing Strings

Creating a String

Strings in PHP can be created using:

- **Single quotes (' ')** (faster, does not parse variables)
- **Double quotes (" ")** (parses variables)

Example: Using Single and Double Quotes

```
<?php
$name = "John Doe";

echo 'Hello, $name'; // Output: Hello, $name (does not parse variable)
echo "<br>";
echo "Hello, $name"; // Output: Hello, John Doe (parses variable)
?>
```

2. Accessing Characters in a String

You can access individual characters in a string using array-style indexing.

Example: Accessing String Characters

```
<?php
$str = "Hello";
echo $str[0]; // Output: H
echo "<br>";
echo $str[4]; // Output: o
?>
```

3. String Concatenation

You can concatenate (combine) strings using the . operator.

Example:

```
<?php
$firstName = "John";
$lastName = "Doe";

$fullName = $firstName . " " . $lastName;
echo $fullName; // Output: John Doe
```

?>

4. String Functions in PHP

PHP provides various string functions to manipulate and modify strings.

a) Finding String Length

```
<?php
$str = "Hello, World!";
echo strlen($str); // Output: 13
?>
```

b) Counting Words in a String

```
<?php
$str = "PHP is a powerful scripting language";
echo str_word_count($str); // Output: 6
?>
```

c) Reversing a String

```
<?php
$str = "Hello";
echo strrev($str); // Output: olleH
?>
```

d) Finding a Substring Position

```
<?php
$str = "Welcome to PHP";
echo strpos($str, "PHP"); // Output: 11 (index starts from 0)
?>
```

e) Replacing a Word in a String

```
<?php
$str = "I love PHP";
echo str_replace("PHP", "Python", $str); // Output: I love Python
?>
```

f) Converting to Upper and Lower Case

```
<?php
$str = "Hello World";

echo strtoupper($str); // Output: HELLO WORLD
echo "<br>";
echo strtolower($str); // Output: hello world
?>
```

5. Extracting a Substring

The substr() function extracts part of a string.

Example:

```
<?php
$str = "Hello, World!";
echo substr($str, 7, 5); // Output: World
?>
```

6. Trimming Spaces from a String

- trim() – Removes spaces from both sides.
- ltrim() – Removes spaces from the left side.
- rtrim() – Removes spaces from the right side.

Example:

```
<?php
$str = " Hello World! ";
echo trim($str); // Output: "Hello World!"
?>
```

7. Checking if a String Contains a Word

You can use strpos() to check if a substring exists.

Example:

```
<?php
$str = "Learn PHP Programming";
if (strpos($str, "PHP") !== false) {
    echo "PHP found in the string!";
} else {
    echo "PHP not found.";
}
?>
```

8. Splitting a String into an Array

The explode() function splits a string based on a delimiter.

Example:

```
<?php
$str = "Apple, Banana, Cherry";
$fruits = explode(", ", $str);

print_r($fruits);
```

```
?>
```

Output:

```
Array ( [0] => Apple [1] => Banana [2] => Cherry )
```

9. Joining an Array into a String

The implode() function joins array elements into a string.

Example:

```
<?php
$fruits = array("Apple", "Banana", "Cherry");
$str = implode(" | ", $fruits);

echo $str; // Output: Apple | Banana | Cherry
?>
```

10. Formatting a String Using sprintf()

The sprintf() function formats a string and returns it.

Example:

```
<?php
$price = 50;
echo sprintf("The price is $%.2f", $price); // Output: The price is $50.00
?>
```

String Manipulation in PHP: Searching, Replacing, Splitting, and Joining

PHP provides powerful functions to search, replace, split, join, and manipulate strings efficiently. Below are examples and explanations of key string operations.

1. Searching for a Substring

The strpos() function is used to find the position of a substring in a string. It returns the position (index) of the first occurrence or false if the substring is not found.

Example: Searching for a Word

```
<?php
$str = "Welcome to PHP programming!";
$search = "PHP";

$position = strpos($str, $search);

if ($position !== false) {
    echo "'$search' found at position $position";
}
```

```

} else {
    echo "'$search' not found in the string.";
}
?>

```

Output:

'PHP' found at position 11

Other Search Functions:

- `strrpos($str, $search)` – Finds the last occurrence of a substring.
- `stripos($str, $search)` – Case-insensitive search.
- `strstr($str, $search)` – Returns the part of the string starting from the search term.

2. Replacing a Substring

The `str_replace()` function replaces occurrences of a substring with another substring.

Example: Replacing a Word

```

<?php
$str = "I love PHP!";
$newStr = str_replace("PHP", "Python", $str);
echo $newStr;
?>

```

Output:

I love Python!

Other Replace Functions:

- `str_ireplace()` – Case-insensitive replacement.
- `substr_replace($str, "replacement", start, length)` – Replaces part of a string.

3. Splitting a String into an Array

The `explode()` function splits a string into an array based on a delimiter.

Example: Splitting a String

```

<?php
$str = "Apple, Banana, Cherry, Mango";
$fruits = explode(",", $str);

print_r($fruits);
?>

```

Output:

Array ([0] => Apple [1] => Banana [2] => Cherry [3] => Mango)

Other Split Functions:

- `str_split($str, length)` – Splits a string into chunks of specified length.

Example: Splitting into 2-character chunks

```
<?php
$str = "Hello";
$chunks = str_split($str, 2);
print_r($chunks);
?>
```

Output:

Array ([0] => He [1] => ll [2] => o)

4. Joining an Array into a String

The `implode()` (or `join()`) function combines array elements into a string using a separator.

Example: Joining an Array

```
<?php
$fruits = array("Apple", "Banana", "Cherry", "Mango");
$str = implode(" | ", $fruits);

echo $str;
?>
```

Output:

Apple | Banana | Cherry | Mango

5. Useful String Functions in PHP

PHP provides numerous built-in string functions. Here are some of the most commonly used ones:

a) Finding String Length

```
<?php
$str = "Hello, World!";
echo strlen($str); // Output: 13
?>
```

b) Counting Words in a String

```
<?php
```



```
$str = "PHP is a powerful scripting language.";
echo str_word_count($str); // Output: 6
?>
```

c) Reversing a String

```
<?php
$str = "Hello";
echo strrev($str); // Output: olleH
?>
```

d) Changing Case of a String

```
<?php
$str = "Hello World";

echo strtoupper($str); // Output: HELLO WORLD
echo "<br>";
echo strtolower($str); // Output: hello world
?>
```

e) Extracting a Substring

```
<?php
$str = "Welcome to PHP";
echo substr($str, 11, 3); // Output: PHP
?>
```

f) Trimming White Spaces

```
<?php
$str = " Hello World! ";
echo trim($str); // Output: "Hello World!"
?>
```

6. Formatting Strings with sprintf()

The sprintf() function formats a string.

Example: Formatting Numbers

```
<?php
$price = 50;
echo sprintf("The price is $%.2f", $price);
?>
```

Output:

The price is \$50.00

7. Checking if a String Starts or Ends with a Substring

PHP does not have built-in functions for this, but you can use strpos() and substr().

Example: Check if a String Starts with a Specific Word

```
<?php
$str = "Hello, welcome to PHP!";
if (strpos($str, "Hello") === 0) {
    echo "The string starts with 'Hello'.";
}
?>
```

Example: Check if a String Ends with a Specific Word

```
<?php
$str = "Hello, welcome to PHP!";
if (substr($str, -3) === "PHP") {
    echo "The string ends with 'PHP'.";
}
?>
```

Handling Sessions and Cookies in PHP

PHP provides **sessions** and **cookies** for managing user data across different pages. Sessions store data on the server, while cookies store data on the user's browser.

1. PHP Sessions

A **session** is a way to store user information (e.g., login data) across multiple pages. The session data is stored on the server.

1.1 Starting a Session

You must call `session_start()` before using sessions.

Example: Creating a Session

```
<?php
session_start(); // Start the session

$_SESSION["username"] = "JohnDoe";
$_SESSION["role"] = "Admin";

echo "Session variables are set.";
?>
```

1.2 Accessing Session Data

You can retrieve session variables on any page where `session_start()` is called.

Example: Accessing Session Data

```
<?php
session_start(); // Start the session

echo "Welcome, " . $_SESSION["username"] . "!<br>";
echo "Your role is: " . $_SESSION["role"];
?>
```

Output:

```
Welcome, JohnDoe!
Your role is: Admin
```

1.3 Destroying a Session

To end a session and clear stored data:

Example: Destroying a Session

```
<?php
```

```
session_start(); // Start the session

session_unset(); // Unset all session variables
session_destroy(); // Destroy the session

echo "Session destroyed.";
?>
```

2. PHP Cookies

A **cookie** is a small file stored on the user's computer. PHP can set and read cookies to remember user preferences.

2.1 Setting a Cookie

The `setcookie()` function is used to create a cookie.

Example: Creating a Cookie

```
<?php
$cookie_name = "user";
$cookie_value = "JohnDoe";
setcookie($cookie_name, $cookie_value, time() + (86400 * 7), "/"); // 7-day expiry

echo "Cookie named '$cookie_name' has been set.";
?>
```

2.2 Accessing a Cookie

Use the `$_COOKIE` superglobal to retrieve cookie values.

Example: Reading a Cookie

```
<?php
if(isset($_COOKIE["user"])) {
    echo "User: " . $_COOKIE["user"];
} else {
    echo "No cookie found.";
}
?>
```

Output:

User: JohnDoe

2.3 Deleting a Cookie

To delete a cookie, set its expiry time to the past.

Example: Deleting a Cookie

```
<?php
setcookie("user", "", time() - 3600, "/"); // Expire 1 hour ago
echo "Cookie deleted.";
?>
```

3. Difference Between Sessions and Cookies

Feature	Sessions	Cookies
Storage	Server-side	Client-side (browser)
Lifetime	Until session is closed or expired	Until manually deleted or expires
Security	More secure	Less secure (visible to users)
Size	No storage limit (depends on server)	Limited to 4KB

4. Practical Example: Login System Using Sessions and Cookies

This example demonstrates how to log in a user using **sessions** and **cookies**.

Step 1: Create login.php

```
<?php
session_start();

if($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = $_POST["username"];
    $password = $_POST["password"];

    if ($username == "admin" && $password == "1234") {
        $_SESSION["user"] = $username;
        setcookie("user", $username, time() + (86400 * 7), "/"); // 7-day cookie
        header("Location: welcome.php");
        exit();
    } else {
        echo "Invalid credentials!";
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
</head>
<body>
    <form method="post">
        Username: <input type="text" name="username" required><br>
        Password: <input type="password" name="password" required><br>
        <input type="submit" value="Login">
    </form>
```

```
</body>
</html>
```

Step 2: Create welcome.php (Dashboard)

```
<?php
session_start();

if (!isset($_SESSION["user"])) {
    header("Location: login.php");
    exit();
}

echo "Welcome, " . $_SESSION["user"] . "!"<br>";

if (isset($_COOKIE["user"])) {
    echo "Remembered user: " . $_COOKIE["user"];
}
?>

<br><a href='logout.php'>Logout</a>
```

Step 3: Create logout.php

```
<?php
session_start();
session_unset();
session_destroy();
setcookie("user", "", time() - 3600, "/"); // Expire cookie

header("Location: login.php");
exit();
?>
```

PHP GET and POST Methods

PHP provides two primary ways to collect form data from users:

GET Method – Sends data via URL parameters.

POST Method – Sends data securely in the request body.

1. Difference Between GET and POST

Feature	GET Method	POST Method
Data Visibility	Data is visible in the URL	Data is hidden from the URL
Security	Less secure (easily visible in browser history, logs)	More secure (not stored in history)
Data Length	Limited by the URL length (about 2000 characters)	No limit on data size
Use Case	When bookmarking/sharing URLs is needed	When sending sensitive data (passwords, login forms)
Form Encoding	Default method for URLs	Used in form submissions

2. Using the GET Method

The `$_GET` superglobal is used to collect data sent via the URL.

Example: Passing Data via URL

Create a file `get_example.php`:

```
<?php
if (isset($_GET["name"]) && isset($_GET["age"])) {
    echo "Hello, " . htmlspecialchars($_GET["name"]) . "<br>";
    echo "You are " . (int)$_GET["age"] . " years old.";
} else {
    echo "No data received.";
}
?>
```

Access it via URL:

`http://yourdomain.com/get_example.php?name=John&age=25`

Output:

Hello, John!
You are 25 years old.

Example: GET with HTML Form

```

<!DOCTYPE html>
<html>
<head><title>GET Example</title></head>
<body>
  <form action="get_example.php" method="GET">
    Name: <input type="text" name="name"><br>
    Age: <input type="number" name="age"><br>
    <input type="submit" value="Submit">
  </form>
</body>
</html>

```

3. Using the POST Method

The `$_POST` superglobal is used to collect form data securely.

Example: Handling POST Request

Create a file `post_example.php`:

```

<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = htmlspecialchars($_POST["name"]);
    $age = (int)$_POST["age"];

    echo "Hello, " . $name . "!<br>";
    echo "You are " . $age . " years old.";
} else {
    echo "No data received.";
}
?>

```

Example: POST with HTML Form

```

<!DOCTYPE html>
<html>
<head><title>POST Example</title></head>
<body>
  <form action="post_example.php" method="POST">
    Name: <input type="text" name="name"><br>
    Age: <input type="number" name="age"><br>
    <input type="submit" value="Submit">
  </form>
</body>
</html>

```

4. Security Considerations

- Use `htmlspecialchars()` to prevent XSS (Cross-Site Scripting).
- Validate input data before using it in queries.

- Use **POST** for sensitive data like passwords.

5. When to Use GET vs. POST

Scenario	Recommended Method
Search queries	GET (so URLs can be shared)
Login forms	POST (to keep passwords secure)
Updating database records	POST (to prevent accidental resubmission)
Retrieving public data	GET (for caching and bookmarking)